

Universidad Autónoma de Santo Domingo

PRIMADA DE AMERICA / Fundada el 28 de octubre de 1538

Nombre:

Wilmy.

Apellidos:

Medina Batista.

Matrícula:

100681393.

Asignatura:

Laboratorio De Lenguaje De Programación III.

Tema:

Modelo Vista Controlador (MVC).

Sección:

01.

Profesor:

Radhames Silverio González.

Fecha:

31/10/2026.

ÍNDICE

INTRODUCCIÓN 3

DESARROLLO: 4

1. ¿Qué es el patrón MVC? 4

2. ¿Cuál es el origen del modelo MVC y para qué fue creado? 4

3. Explicación de cada componente del modelo: Modelo, Vista y Controlador. 5

4. Ventajas y desventajas del uso de MVC. 6

Ventajas: 6

Desventajas: 6

5. ¿Por qué MVC sigue siendo importante en la programación actual? 7

6. ¿Cómo se usa MVC hoy en día en frameworks modernos? 8

7. Comparar el MVC tradicional con la forma en que se aplica actualmente en Spring MVC, ASP.NET MVC, Laravel, Django, Ruby on Rails u otros frameworks similares. 8

Comparación general: 10

8. Incluir un ejemplo sencillo donde se identifique claramente el Modelo, la Vista y el Controlador. 10

En este ejemplo, la separación de responsabilidades sería la siguiente: 10

Recorrido de una operación: 11

Modelo (Estudiante.php): 11

Controlador (EstudianteController.php): 11

Vista (registro.php): 12

Explicación del funcionamiento: 13

9. Conclusión personal sobre la importancia de comprender MVC antes de trabajar con frameworks modernos. 14

CONCLUSIÓN 15

REFERENCIAS BIBLIOGRÁFICAS 16

INTRODUCCIÓN

En el desarrollo de aplicaciones de software, una de las decisiones más importantes consiste en organizar el código de manera que cada parte tenga una responsabilidad definida. Cuando una aplicación crece, colocar en un mismo lugar las reglas del negocio, el acceso a los datos y la presentación de la información puede dificultar su mantenimiento. Para enfrentar este tipo de problemas se utilizan patrones de arquitectura y diseño que ayudan a distribuir las tareas dentro de un sistema. Uno de los más conocidos es el Modelo-Vista-Controlador, identificado por las siglas MVC.

El patrón MVC propone separar una aplicación en tres componentes que colaboran entre sí: el Modelo, que representa los datos y las reglas de la aplicación; la Vista, que se encarga de presentar la información; y el Controlador, que recibe las acciones o solicitudes y coordina la respuesta del sistema. Esta división permite comprender mejor el funcionamiento de un programa y facilita que sus componentes puedan modificarse con menor dependencia entre ellos.

El origen de MVC se relaciona con el trabajo de Trygve Reenskaug en el entorno Smalltalk del Xerox PARC, a finales de la década de 1970. La idea surgió para organizar interfaces gráficas interactivas, separando la representación de los datos de la forma en que el usuario los observaba y manipulaba. Con el paso del tiempo, el patrón fue adaptado al desarrollo de aplicaciones de escritorio, sistemas empresariales y, posteriormente, aplicaciones web.

En la actualidad, MVC sigue siendo una referencia importante, aunque los frameworks no siempre lo implementan de manera idéntica. Tecnologías como Spring MVC, ASP.NET Core MVC, Laravel y Ruby on Rails utilizan conceptos relacionados con esta separación. Django, por su parte, aplica una organización conocida como Modelo-Plantilla-Vista (MTV), que mantiene objetivos semejantes, pero asigna nombres y responsabilidades de una manera particular. Por esta razón, conocer el MVC tradicional ayuda a comprender las decisiones de diseño que se encuentran detrás de muchas herramientas modernas.

El propósito de esta investigación es explicar qué es MVC, cómo se originó, cuál es la función de cada uno de sus componentes y qué ventajas y limitaciones presenta. También se analiza por qué continúa siendo relevante, cómo aparece en distintos frameworks actuales y cómo podría organizarse un sistema sencillo de registro de estudiantes siguiendo este patrón.

DESARROLLO:

1. ¿Qué es el patrón MVC?

El Modelo-Vista-Controlador (MVC) es un patrón de arquitectura de software que organiza una aplicación en tres responsabilidades principales: administrar la información y las reglas del sistema, presentar los datos al usuario y gestionar las interacciones que conectan ambas partes. Su finalidad es evitar que toda la lógica de una aplicación quede concentrada en un único bloque de código.

En este patrón, el Modelo representa el estado de la aplicación y las operaciones que se realizan sobre los datos. La Vista muestra la información de una forma comprensible, por ejemplo, mediante una página web, una tabla o un formulario. El Controlador recibe una acción del usuario o una solicitud, interpreta lo que se necesita y coordina el trabajo entre el Modelo y la Vista.

MVC no es un lenguaje de programación ni un framework específico. Es una forma de organizar el diseño de un sistema. Puede implementarse con distintos lenguajes y herramientas, siempre que se mantenga una separación razonable entre los datos, la presentación y la coordinación de las acciones. Aunque cada framework puede adaptar el patrón, la idea central continúa siendo dividir responsabilidades para que el software sea más fácil de entender, probar y mantener (Reenskaug, 1979; Microsoft, s. f.-a).

Es importante señalar que MVC no exige que los tres componentes sean archivos únicos ni que exista una sola manera de comunicarlos. En una aplicación real, el Modelo puede estar formado por varias clases, servicios y repositorios; la Vista puede incluir plantillas y componentes visuales; y el Controlador puede contener diferentes acciones que responden a rutas o eventos.

2. ¿Cuál es el origen del modelo MVC y para qué fue creado?

El origen de MVC se encuentra en los trabajos de Trygve Reenskaug durante su participación en el proyecto Smalltalk del Xerox Palo Alto Research Center (Xerox PARC), en la década de 1970. Reenskaug buscaba una forma de organizar las interfaces de usuario interactivas para que los datos de una aplicación no dependieran directamente de una única representación visual. Sus ideas fueron documentadas en 1979 y se convirtieron en una base para el patrón que posteriormente se conocería como Model-View-Controller (Reenskaug, 1979).

El propósito inicial era facilitar la construcción de programas interactivos, especialmente aquellos en los que el usuario podía observar y modificar información mediante una interfaz gráfica. En vez de mezclar los datos, la presentación y las acciones en un solo objeto o bloque de código, se propuso distribuir esas funciones. De esa manera, una misma

información podía presentarse de distintas formas y la interfaz podía cambiar sin tener que reconstruir toda la lógica que administraba los datos.

En su contexto original, MVC estaba relacionado con el desarrollo de interfaces gráficas y no fue creado específicamente para páginas web, porque la Web todavía no existía como plataforma de aplicaciones. Con la evolución de la programación orientada a objetos y de las tecnologías web, el patrón se adaptó a nuevos entornos. Los frameworks web incorporaron controladores que reciben peticiones HTTP, modelos que representan datos y vistas que generan contenido para el navegador.

Por tanto, MVC nació como una propuesta para ordenar la interacción entre una persona y un programa. Su uso posterior en aplicaciones web es una evolución del concepto original, no su propósito inicial. Esta distinción permite comprender por qué algunas implementaciones actuales no reproducen exactamente la comunicación de los primeros sistemas Smalltalk.

3. Explicación de cada componente del modelo: Modelo, Vista y Controlador.

Los tres componentes de MVC tienen funciones diferentes, pero trabajan de forma coordinada. La separación no significa que cada uno funcione de manera aislada; significa que cada parte debe concentrarse principalmente en una responsabilidad.

Modelo (Model):

El Modelo representa los datos que utiliza la aplicación y las reglas que determinan cómo esos datos pueden consultarse o modificarse. En un sistema de estudiantes, por ejemplo, el modelo puede representar la matrícula, el nombre, la carrera y el correo electrónico de cada estudiante. También puede incluir operaciones para registrar un estudiante, buscarlo, actualizar sus datos o comprobar ciertas condiciones.

El Modelo no debe depender de la forma exacta en que la información se muestra en pantalla. Su función es mantener la información y las reglas del dominio de la aplicación. En proyectos más grandes, esta responsabilidad puede distribuirse entre entidades, servicios de negocio y componentes de acceso a datos.

Vista (View):

La Vista es la parte encargada de presentar la información al usuario. Puede ser una página HTML, una plantilla, una pantalla de escritorio o una interfaz compuesta por diferentes elementos visuales. Su responsabilidad principal consiste en mostrar los datos recibidos y ofrecer controles para que el usuario pueda interactuar con el sistema.

En el ejemplo de registro de estudiantes, la Vista puede ser un formulario con campos para matrícula, nombre, carrera y correo. También puede existir otra vista que presente una tabla con los estudiantes registrados. La Vista debe concentrarse en la presentación y evitar asumir responsabilidades que corresponden a las reglas del negocio.

Controlador (Controller):

El Controlador actúa como intermediario entre las solicitudes del usuario, el Modelo y la Vista. Recibe una acción, identifica qué operación se necesita, obtiene o envía información al Modelo y determina cuál será la respuesta. En una aplicación web puede corresponder a una clase con métodos asociados a rutas, como mostrar un formulario, guardar un registro o consultar un estudiante.

Por ejemplo, cuando una persona envía el formulario de registro, el Controlador recibe los campos, comprueba que la solicitud tenga la información necesaria y solicita al Modelo que registre los datos. Si la operación se completa, puede seleccionar una Vista de confirmación; si existen errores, puede devolver el formulario con los mensajes correspondientes. Las validaciones importantes deben realizarse también en el servidor y no confiar únicamente en los controles del navegador.

En resumen, el Modelo administra los datos y las reglas, la Vista presenta la información y el Controlador coordina las solicitudes y las respuestas. La colaboración entre estos componentes permite que una aplicación mantenga una estructura más comprensible.

4. Ventajas y desventajas del uso de MVC.

Ventajas:

- **Separación de responsabilidades:** Cada componente tiene una función principal, lo que reduce la mezcla entre la lógica del negocio, la presentación y el manejo de solicitudes.
- **Mantenimiento más sencillo:** Cuando se modifica una parte de la aplicación, la separación puede reducir la cantidad de componentes que necesitan cambiar al mismo tiempo.
- **Reutilización de componentes:** Un mismo Modelo puede proporcionar datos a diferentes Vistas, como una página de consulta y un panel administrativo.
- **Facilidad para realizar pruebas:** La lógica del Modelo y las acciones del Controlador pueden probarse de forma más organizada, sin depender siempre de la interfaz completa.
- **Trabajo en equipo:** La separación permite que diferentes desarrolladores trabajen en la presentación, la lógica y el manejo de solicitudes con acuerdos claros.
- **Organización del proyecto:** El patrón ofrece una estructura reconocible que facilita localizar archivos y comprender el recorrido de una operación.

Desventajas:

- **Mayor estructura inicial:** En una aplicación pequeña, crear controladores, modelos y vistas separados puede parecer más trabajo que resolver todo en pocos archivos.

- **Complejidad en proyectos grandes:** Si no se establecen límites claros, pueden aparecer controladores demasiado extensos, modelos con demasiadas responsabilidades o dependencias difíciles de mantener.
- **Curva de aprendizaje:** Quien comienza a programar debe comprender cómo se comunican los componentes y dónde debe ubicarse cada operación.
- **Implementaciones diferentes:** Los frameworks no aplican MVC exactamente de la misma manera, por lo que trasladar conocimientos de una herramienta a otra puede requerir ajustes.
- **Riesgo de sobre arquitectura:** Aplicar el patrón de manera rígida a problemas sencillos puede introducir capas y archivos que no aportan un beneficio proporcional.
- **No resuelve todos los problemas:** MVC organiza responsabilidades, pero no garantiza por sí solo seguridad, rendimiento, escalabilidad ni calidad del código.

En consecuencia, MVC es útil cuando la separación facilita el mantenimiento y la comprensión del sistema. Sin embargo, debe aplicarse de acuerdo con el tamaño y las necesidades del proyecto, evitando agregar complejidad innecesaria.

5. ¿Por qué MVC sigue siendo importante en la programación actual?

MVC continúa siendo importante porque la separación de responsabilidades sigue siendo una necesidad en el desarrollo de software. Aunque las herramientas y los lenguajes han cambiado, las aplicaciones todavía necesitan administrar información, mostrarla y responder a las acciones de los usuarios. Organizar esas funciones por separado ayuda a que los proyectos puedan evolucionar sin convertir el código en una estructura difícil de comprender.

También es importante porque muchos frameworks y plataformas utilizan MVC de manera directa o se inspiran en sus principios. Comprender el patrón permite reconocer dónde se procesa una solicitud, dónde se encuentran las reglas de negocio y cómo se genera la respuesta que recibe el usuario. De esta forma, aprender un framework no consiste solamente en memorizar comandos o convenciones, sino en entender la arquitectura que organiza el proyecto.

Otro motivo es su valor educativo. MVC ayuda a desarrollar hábitos de diseño como la cohesión, el bajo acoplamiento y la asignación clara de responsabilidades. Estos principios no pertenecen exclusivamente a MVC; aparecen en otras arquitecturas, como las aplicaciones por capas, la arquitectura limpia y los sistemas basados en servicios.

Se considera un modelo antiguo porque sus orígenes se remontan a finales de los años setenta. Sin embargo, antiguo no significa obsoleto. MVC continúa siendo útil porque resuelve un problema que sigue presente: cómo organizar la interacción entre datos, interfaz y acciones. Lo que ha cambiado es la forma de implementarlo, los protocolos utilizados y la cantidad de componentes que pueden intervenir en una aplicación moderna.

6. ¿Cómo se usa MVC hoy en día en frameworks modernos?

En las aplicaciones web actuales, MVC suele adaptarse al ciclo de petición y respuesta HTTP. El navegador solicita una dirección; el framework dirige esa solicitud hacia un controlador; el controlador ejecuta o delega la operación necesaria; el Modelo participa en la consulta o modificación de los datos; y finalmente se devuelve una Vista HTML o, en determinados casos, una respuesta en formato JSON u otro contenido.

Los frameworks modernos proporcionan mecanismos para facilitar este trabajo, como enrutamiento, validación de datos, acceso a bases de datos, plantillas, filtros, autenticación y manejo de errores. Así, el desarrollador no necesita construir desde cero todas las piezas que conectan una solicitud con la lógica de la aplicación.

En muchos proyectos actuales, la Vista puede estar en una aplicación de frontend independiente y el backend puede responder mediante una API. En ese escenario, el servidor conserva parte de la organización de modelos y controladores, pero la presentación se realiza en otro proyecto o aplicación. Por eso, MVC se utiliza tanto en aplicaciones que generan páginas desde el servidor como en sistemas que ofrecen servicios a clientes web o móviles.

Además, los frameworks pueden incorporar servicios, repositorios, middleware y capas adicionales. Estas piezas no eliminan necesariamente MVC; suelen complementar la estructura para mantener los controladores pequeños, separar el acceso a datos y facilitar las pruebas. La implementación concreta depende de las convenciones de cada tecnología.

7. Comparar el MVC tradicional con la forma en que se aplica actualmente en Spring MVC, ASP.NET MVC, Laravel, Django, Ruby on Rails u otros frameworks similares.

El MVC tradicional presenta tres componentes conceptuales: Modelo, Vista y Controlador. En los frameworks actuales se conserva la idea de separar responsabilidades, pero cada tecnología organiza las clases, rutas y plantillas de acuerdo con sus propias convenciones. Por eso, los nombres pueden variar y algunas responsabilidades pueden dividirse en servicios o componentes adicionales.

Spring MVC:

Spring MVC es un framework web de Java que utiliza el patrón Modelo-Vista-Controlador para manejar solicitudes HTTP. Los controladores se definen normalmente mediante anotaciones como @Controller o @RestController, y sus métodos responden a rutas específicas. En una aplicación que genera páginas, el controlador puede devolver el nombre de una vista y proporcionar datos para renderizarla. En una API, un controlador REST puede devolver directamente datos, por ejemplo, en JSON. La documentación de Spring describe este modelo de solicitudes y controladores como parte de su infraestructura web (VMware, s. f.).

ASP.NET Core MVC:

ASP.NET Core MVC es una plataforma de Microsoft para crear aplicaciones web siguiendo el patrón MVC. Las solicitudes se asignan a controladores y acciones; los modelos representan los datos que se reciben o procesan; y las vistas Razor permiten generar páginas HTML. El framework también ofrece model binding, validación y enrutamiento. Una diferencia importante es que ASP.NET Core MVC integra estas funciones dentro del ecosistema de ASP.NET Core, por lo que puede combinarse con middleware, inyección de dependencias y otros servicios de la plataforma (Microsoft, s. f.-a; Microsoft, s. f.-b).

Laravel:

Laravel es un framework PHP que organiza las aplicaciones mediante rutas, controladores, modelos Eloquent y vistas Blade. Las rutas determinan qué acción se ejecuta ante una solicitud; los controladores reúnen la lógica necesaria para responder; los modelos Eloquent representan registros y permiten trabajar con la base de datos; y las vistas Blade construyen la presentación. Laravel mantiene una estructura muy cercana al enfoque MVC, aunque permite incorporar servicios y otras clases para no concentrar toda la lógica en los controladores (Laravel, s. f.-a; Laravel, s. f.-b).

Django:

Django utiliza una organización que denomina Modelo-Plantilla-Vista (MTV). El Modelo representa los datos y se relaciona con el sistema de persistencia; la Plantilla define cómo se presenta el contenido; y la Vista de Django recibe la solicitud, ejecuta la lógica correspondiente y devuelve una respuesta. Esta Vista cumple una función que en la terminología MVC tradicional suele asociarse principalmente con el Controlador. Django también dispone de un sistema de URLconf que conecta las direcciones con las vistas. Por ello, puede considerarse conceptualmente relacionado con MVC, pero no es correcto afirmar que utiliza exactamente los mismos nombres o la misma distribución de responsabilidades (Django Software Foundation, s. f.).

Ruby on Rails:

Ruby on Rails es un framework web que promueve convenciones para organizar aplicaciones en modelos, vistas y controladores. Los modelos suelen trabajar con Active Record para representar datos y relacionarlos con la base de datos; los controladores reciben las solicitudes y seleccionan la respuesta; y las vistas presentan la información mediante plantillas. Rails automatiza buena parte de la conexión entre rutas, controladores y vistas, por lo que el desarrollador puede concentrarse en las reglas de la aplicación siguiendo convenciones establecidas (Rails, s. f.-a; Rails, s. f.-b).

Comparación general:

Tecnología	Organización principal	Cómo aplica la idea	Rasgo distintivo
MVC tradicional	Modelo, Vista y Controlador	Separa datos/reglas, presentación y coordinación de acciones.	Patrón conceptual, no framework.
Spring MVC	Controladores, modelos, vistas o respuestas REST	Mapea solicitudes HTTP a métodos de controlador y genera vistas o datos.	Integración con el ecosistema Java y Spring.
ASP.NET Core MVC	Modelos, controladores, acciones y vistas Razor	Usa enrutamiento, model binding y vistas para responder solicitudes.	Integración con ASP.NET Core y Razor.
Laravel	Rutas, controladores, Eloquent y Blade	Conecta rutas con controladores, modelos y plantillas.	Convenciones PHP y ORM Eloquent.
Django (MTV)	Modelo, Plantilla y Vista	La vista recibe la solicitud y coordina la respuesta; la plantilla presenta.	Nombres y responsabilidades diferentes a MVC clásico.
Ruby on Rails	Modelos, vistas y controladores	Convenciones conectan rutas, acciones y plantillas.	Convención sobre configuración y Active Record.

La comparación permite observar que Spring MVC, ASP.NET Core MVC, Laravel y Ruby on Rails mantienen una relación directa con el patrón MVC, aunque cada uno añade herramientas y convenciones propias. Django conserva objetivos similares, pero utiliza el nombre MTV y asigna a sus vistas una responsabilidad que se aproxima al controlador del modelo tradicional. En todos los casos, conocer la idea general de MVC facilita comprender la estructura de los proyectos, pero es necesario estudiar las reglas particulares de cada framework.

8. Incluir un ejemplo sencillo donde se identifique claramente el Modelo, la Vista y el Controlador.

Para aplicar los conceptos anteriores, se propone un sistema de registro de estudiantes. El objetivo es permitir que una persona complete un formulario con la matrícula, el nombre, la carrera y el correo electrónico. Cuando envía la información, el sistema valida los campos, guarda el registro y presenta un mensaje de confirmación.

En este ejemplo, la separación de responsabilidades sería la siguiente:

- **Modelo:** La clase Estudiante representa la información de cada estudiante, mientras que un servicio o repositorio se encarga de guardar y consultar los registros. El Modelo contiene las reglas necesarias para que los datos mantengan una estructura válida.
- **Vista:** El formulario HTML solicita matrícula, nombre, carrera y correo. Otra vista puede mostrar una lista de estudiantes registrados o un mensaje indicando que el registro fue completado.
- **Controlador:** La acción registrar recibe los datos enviados por el formulario, comprueba que sean válidos, solicita al Modelo guardar el estudiante y selecciona la respuesta que se mostrará.

Recorrido de una operación:

1. **Presentación del formulario:** El usuario abre la página y el Controlador devuelve la Vista que contiene el formulario.
2. **Envío de datos:** El usuario completa los campos y presiona el botón de registrar. El navegador envía una solicitud HTTP al servidor.
3. **Recepción y validación:** El Controlador recibe los valores y comprueba que los campos requeridos tengan información y un formato adecuado.
4. **Operación del Modelo:** Si los datos son válidos, el Controlador solicita al Modelo o al servicio correspondiente que cree y almacene el estudiante.
5. **Respuesta:** El Controlador devuelve una Vista de confirmación o el formulario con los errores que deben corregirse.

A continuación se muestra un ejemplo simplificado con PHP y una vista HTML. Se utiliza una lista en memoria para concentrarse en la interacción entre los componentes; por tanto, los registros no se conservan después de finalizar la ejecución. En un sistema real, el Modelo se conectaría a una base de datos y se aplicarían medidas adicionales de seguridad.

Modelo (Estudiante.php):

```
<?php
class Estudiante {
public string $matricula;
public string $nombre;
public string $carrera;
public string $correo;

public function __construct($matricula, $nombre, $carrera, $correo) {
$this->matricula = $matricula;
$this->nombre = $nombre;
$this->carrera = $carrera;
$this->correo = $correo;


}


}
```

Controlador (EstudianteController.php):

```
<?php
require_once "Estudiante.php";

class EstudianteController {
public function registrar() {
if ($_SERVER["REQUEST_METHOD"] !== "POST") {
```

```
return null;


}

$matricula = trim($_POST["matricula"] ?? "");
$nombre = trim($_POST["nombre"] ?? "");
$carrera = trim($_POST["carrera"] ?? "");
$correo = trim($_POST["correo"] ?? "");

if ($matricula === "" || $nombre === "" ||
$carrera === "" || !filter_var($correo, FILTER_VALIDATE_EMAIL)) {
return "Revise los datos ingresados.";


}

$estudiante = new Estudiante(
$matricula, $nombre, $carrera, $correo


);

// En una aplicación real se guardaría en una base de datos.
return "Registro recibido para: " . $estudiante->nombre;


}


}
```

Vista (registro.php):

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Registro de estudiantes</title>
</head>
<body>
<h1>Registro de estudiantes</h1>

<form method="POST" action="procesar.php">
<label>Matrícula:</label>
<input type="text" name="matricula" required>

<label>Nombre:</label>
<input type="text" name="nombre" required>

<label>Carrera:</label>
<input type="text" name="carrera" required>

<label>Correo:</label>
```

```
<input type="email" name="correo" required>

<button type="submit">Registrar estudiante</button>
</form>
</body>
</html>
```

Para completar la demostración, el archivo procesar.php puede crear el Controlador y ejecutar su acción de registro:

```
<?php
require_once "EstudianteController.php";

$controlador = new EstudianteController();
$resultado = $controlador->registrar();

echo htmlspecialchars($resultado ?? "Solicitud no válida");
```

Explicación del funcionamiento:

- **Formulario:** La Vista presenta los campos que el usuario necesita completar y envía los datos mediante el método POST.
- **Controlador:** El archivo procesar.php crea el controlador y llama a la operación registrar(), que recibe y revisa la información.
- **Validación:** El controlador comprueba que matrícula, nombre y carrera no estén vacíos y que el correo tenga un formato válido. La validación del navegador mejora la experiencia, pero la comprobación del servidor es la que debe proteger el procesamiento.
- **Modelo:** Cuando los datos cumplen las condiciones, se crea un objeto Estudiante con la información recibida. En una aplicación completa, el Modelo o una capa de persistencia guardaría ese objeto en una base de datos.
- **Respuesta:** El controlador devuelve un mensaje que se muestra al usuario. En un proyecto más completo, podría redirigir a una vista de confirmación o volver al formulario con mensajes de validación.

Este ejemplo permite identificar la función de cada componente sin depender de un framework específico. También muestra que el Controlador no debería convertirse en el lugar donde se concentra toda la lógica del sistema. A medida que la aplicación crece, las operaciones de negocio y el acceso a datos pueden trasladarse a servicios y repositorios para conservar una separación clara.

9. Conclusión personal sobre la importancia de comprender MVC antes de trabajar con frameworks modernos.

Después de estudiar el patrón Modelo-Vista-Controlador, considero que comprender MVC es una base importante para cualquier persona que se esté preparando en el desarrollo de aplicaciones. Aunque se trata de una arquitectura que tiene varias décadas de existencia, sus principios continúan presentes en muchas herramientas que se utilizan actualmente. Su valor no depende de que sea una tecnología nueva, sino de que propone una forma ordenada de distribuir las responsabilidades de un programa.

Uno de los aspectos que considero más importantes es aprender a distinguir dónde deben ubicarse los datos y las reglas de una aplicación, dónde se presenta la información y quién se encarga de coordinar las acciones del usuario. Cuando se entiende esa diferencia, resulta más sencillo leer un proyecto, localizar errores y realizar modificaciones sin afectar innecesariamente otras partes del sistema.

También considero que estudiar MVC antes de utilizar frameworks como Spring MVC, ASP.NET Core MVC, Laravel o Ruby on Rails ayuda a comprender por qué existen controladores, modelos, vistas, rutas y plantillas. En lugar de memorizar la estructura de una herramienta, el estudiante puede reconocer la idea que hay detrás de ella y adaptarse con mayor facilidad a diferentes tecnologías. Incluso cuando un framework utiliza una organización distinta, como ocurre con el enfoque MTV de Django, los principios de separación de responsabilidades siguen siendo útiles.

En conclusión, MVC no debe verse como una solución obligatoria para todos los proyectos ni como un patrón que haya permanecido sin cambios. Es una referencia histórica y técnica que ha evolucionado junto con el desarrollo de software. Comprenderlo permite construir una base sólida para aprender arquitecturas más modernas, organizar mejor el código y tomar decisiones conscientes al desarrollar aplicaciones web y servicios.

CONCLUSIÓN

El Modelo-Vista-Controlador (MVC) representa uno de los patrones de arquitectura más importantes en la historia del desarrollo de software, debido a que estableció una forma organizada de distribuir las responsabilidades dentro de una aplicación. Desde su origen, este patrón fue creado con el propósito de separar la información, su presentación y las acciones que permiten interactuar con ella. Esta división permitió desarrollar sistemas más estructurados y facilitó el mantenimiento y la evolución de los programas.

A lo largo del tiempo, MVC ha experimentado diferentes adaptaciones para responder a las necesidades de las nuevas tecnologías. Su aplicación en frameworks como Spring MVC, ASP.NET Core MVC, Laravel y Ruby on Rails demuestra que sus principios continúan presentes en el desarrollo de aplicaciones web modernas. Aunque algunas tecnologías, como Django, utilizan una organización diferente en cuanto a los nombres y las responsabilidades de sus componentes, mantienen objetivos similares relacionados con la separación de funciones.

Asimismo, este patrón permite comprender cómo se relacionan los datos, las interfaces y las solicitudes dentro de una aplicación. Sus ventajas se reflejan principalmente en la organización del código, la facilidad de mantenimiento, la reutilización de componentes y la posibilidad de realizar pruebas de manera más independiente. Sin embargo, su implementación debe adaptarse a las características de cada proyecto, ya que una estructura demasiado compleja puede generar trabajo innecesario.

En definitiva, MVC continúa siendo un fundamento importante de la ingeniería de software. Aunque han surgido nuevas arquitecturas y herramientas de desarrollo, los principios de organización y separación de responsabilidades que propone siguen teniendo aplicación en los sistemas actuales. Por esta razón, conocer su origen, funcionamiento y evolución permite comprender mejor la arquitectura de las aplicaciones modernas y desarrollar soluciones informáticas más organizadas, mantenibles y eficientes.

REFERENCIAS BIBLIOGRÁFICAS

Django Software Foundation. (s. f.). Request and response objects. Django documentation. https://docs.djangoproject.com/en/stable/ref/request-response/

Laravel. (s. f.-a). Controllers. Laravel documentation. https://laravel.com/docs/controllers

Laravel. (s. f.-b). Eloquent: Getting started. Laravel documentation. https://laravel.com/docs/eloquent

Microsoft. (s. f.-a). Overview of ASP.NET Core MVC. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/mvc/overview

Microsoft. (s. f.-b). Views in ASP.NET Core MVC. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/mvc/views/overview

Rails. (s. f.-a). Getting started with Rails. Ruby on Rails Guides. https://guides.rubyonrails.org/getting_started.html

Rails. (s. f.-b). Action Controller overview. Ruby on Rails Guides. https://guides.rubyonrails.org/action_controller_overview.html

Reenskaug, T. (1979, 10 de diciembre). Models–Views–Controllers. Zenodo. https://zenodo.org/records/3676092/files/1979-12-MVC.pdf

VMware. (s. f.). Web on Servlet stack. Spring Framework reference documentation. https://docs.spring.io/spring-framework/reference/web/webmvc.html